

Д32 — Roguelike с LLM-агентом

Курс: Software Design, ИТМО MSE, весна 2026

Команда: 3–4 человека

Язык и стек: любой

Защита: очно, середина июня

Что нужно сделать

Команда разрабатывает играбельную roguelike-игру с графикой. Игра выставляет свое API наружу через **MCP-сервер**, через который в нее может играть **LLM-агент**. Дополнительно, поведение части мобов также управляется LLM. В финале команда сравнивает 2+ AI-агентов на одинаковых сидах и пишет отчет.

Чекпойнты и дедлайны

Дата	Что сдается
20.05	service diagram + MCP-контракт + docker compose up поднимает 3 (хотя бы заглушечных) сервиса
31.05	игра играется руками; mcp-server отдает корректный state; агент дошел до конца хотя бы раз
10.06	финальная версия в репо: эвал-скрипт, 2+ агента, логи, отчет
сессия (~ середина июня)	очная защита

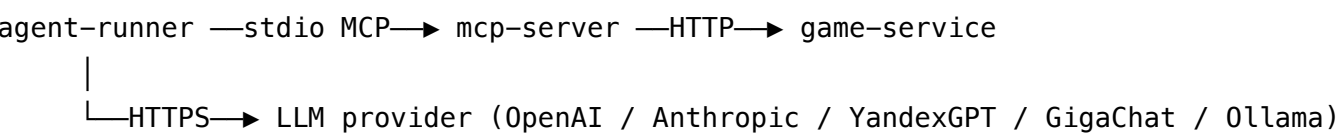
Требования

1. Игра

- **Графика обязательна.** 2D / 3D / web canvas. Не текстовый CLI.
- Минимальная механика:
 - Герой с HP, инвентарем, уровнями
 - Случайно генерируемая карта (комнаты + коридоры)
 - 3+ типа мобов с разным поведением
 - Боевая система: атака, защита, урон, смерть
 - Условие победы и условие проигрыша

2. Архитектура: минимум 3 микросервиса

Это **обязательное требование курса**, не опция.



Сервис	Ответственность
game-service	Игровой движок: state, карта, боевка, mobs
mcp-server	MCP-протокол: tool'ы, JSON-RPC, проксирование к game-service
agent-runner	Agent loop: LLM-клиент, retry, budget, логирование tool calls

Границы ответственности (проверяется на ревью):

- game-service не знает про LLM
- mcp-server не знает про LLM
- agent-runner не знает про внутренний state игры — только через MCP-tool'ы

Деплой: вся система поднимается одной командой `docker compose up` . Ручные шаги между контейнерами не допускаются.

Опциональные сервисы (за бонус): `web-dashboard` , `logger` , `replay-service` , `event-bus` .

3. Паттерны GoF

Минимум **3 паттерна** из: Strategy, State, Factory, Command, Observer.

Каждый описан в README:

- какой класс / модуль
- ссылка `file:line`
- что решает
- почему именно этот паттерн

4. AI-блок

4.1. MCP-сервер

- **5+ tool'ов** с осмысленными именами и описаниями
- **MCP-контракт** в репо (`mcp-contract.md` или `tools.json`): имя, описание, JSON-схема аргументов, пример ответа для каждого tool
- При невалидных аргументах возвращается ошибка с понятным сообщением

4.2. Agent loop

- `agent-runner` подключается к `mcp-server` , получает `tools/list` , играет
- **Минимум 2 LLM-провайдера**, переключаемых через `LLM_PROVIDER=...` (без правки кода)
- **Budget**: жесткий лимит шагов и/или токенов на одну партию
- **Graceful degradation**: при ошибке LLM-провайдера агент не падает (retry с backoff и/или fallback)

4.3. LLM-управляемые враги

Минимум **один тип моба** реально вызывает LLM для принятия решения (через tool use), а не if/else в Strategy.

4.4. Сравнение агентов

- **2+ агента** на одинаковых сидах генерации мира
- **Минимум 5 прогонов на агента**
- Метрики (минимум):
 - % успешных прохождений
 - среднее число шагов
 - среднее число токенов на прогон

- среднее HP в финале (если дошел)
- Отчет в репо: `eval/results.md` с таблицей и графиками

5. Качество

- **Юнит-тесты** на ключевую логику (combat, pathfinding, инвентарь)
- **Тесты agent loop с FakeLLM** — без реальных вызовов API
- **CI** на каждый push: тесты + линтер
- **README** содержит:
 - как запустить (одна команда)
 - переменные окружения
 - как подменить LLM-провайдера
 - **раздел про использование AI** (Claude Code / Copilot / ChatGPT): что генерировали, что правили руками

Что НЕ делать (антипаттерны)

Нарушение	Что значит
Читы агенту	agent-runner имеет прямой доступ к state игры мимо MCP
Хардкод одного провайдера	Подменить на другой нельзя без правки кода
Один прогон	Сравнение агентов на 1–2 запусках без статистики
Один сид	Эксперимент не воспроизводим
Нет budget	Агент может зациклиться
Тесты ходят в реальный LLM	Юнит-тесты должны быть на FakeLLM
Закоммиченные API-ключи	<code>.env</code> с секретами в Git

Все эти пункты влекут штрафные баллы.

Защита

Очно в классе, середина июня (точная дата — в чате курса).

Формат на команду (~15 минут):

- 5–7 мин: презентация (3–5 слайдов)
- 5–7 мин: live demo
- 2–3 мин: Q&A

Что показать:

- Архитектура: service diagram с подписанными транспортами
- Паттерны: какие 3+ применили, где
- AI-блок: какие агенты сравнивали, главный график результатов
- Демка: запуск `docker compose up`, игра руками, агент играет вживую
- Lessons learned

К чему готовиться:

- Записанные видео демки (на случай, если будут проблемы с интернетом в университете)
- Презентация в PDF
- Ноут с поднятой и проверенной системой

Ресурсы

- [MCP spec](#)
- [Anthropic — Building Effective Agents](#)
- [OpenAI Function Calling](#)
- [YandexGPT API](#) — поле `text` вместо `content`, роль `assistant` для tool result

FAQ

Можно ли использовать игровые движки (Pygame / Phaser / Unity)?

Да.

Можно ли использовать LangChain / агент-фреймворки?

Нет. Голый MCP + свой agent loop проще дебажить и лучше вас научит.

Сколько денег на API?

Запросы в GigaChat и YandexGPT стоят довольно дешево, но если не хотите тратить деньги – можно посмотреть в сторону поднятия своей LLM.

Использовать AI для кодинга — ок?

Поощряется. Но в **README честно описать**, что сгенерировали и где правили. Команды, которые не понимают свой код, проваливают Q&A на защите.

Что если не успеваем?

Пишите мне в чат за неделю до чекпойнта — обсудим, что урезать. Лучше сдать урезанное, что работает, чем амбициозное, что не запускается.